Initial System Structure

Initial System Structure

1. Initial System Structure

The system will be divided into four main subsystems:

1. Client Subsystem
2. Backend Subsystem
3. GenAI Subsystem
4. Database Subsystem

Client Subsystem

The Client Subsystem will be implemented using **React**. It provides the user interface for the language learning application.

It allows users to:

- Register and log in
- Set up their language profile
- Choose their target language and learning goal
- View their personalized learning plan
- Solve exercises
- Practice conversations
- View feedback and progress

Backend Subsystem

The Backend Subsystem will be implemented using **Spring Boot REST APIs**. It contains the main application logic and connects the frontend, database, and GenAI service.

The backend will be divided into three Spring Boot microservices:

1. User Service
2. Learning Service
3. Progress & Feedback Service

User Service

The User Service manages all user-related functionality.

It is responsible for:

- User registration
- User login
- User profile management
- Saving the target language
- Saving the current language level
- Saving the learning goal
- Saving the deadline or available study time

Learning Service

The Learning Service manages learning plans, lessons, and exercises.

It is responsible for:

- Creating learning plans
- Managing lessons
- Managing exercises
- Showing daily learning tasks
- Requesting generated plans and exercises from the GenAI Service

Progress & Feedback Service

The Progress & Feedback Service manages user answers, feedback, scores, and progress tracking.

It is responsible for:

- Storing user answers
- Storing feedback results
- Tracking completed exercises
- Tracking user scores
- Identifying weak areas
- Displaying simple progress statistics
- Requesting answer analysis and feedback from the GenAI Service

GenAI Subsystem

The GenAI Subsystem will be implemented as a separate **Python microservice** using **FastAPI and LangChain**.

It is responsible for:

- Generating personalized learning plans
- Generating vocabulary and grammar exercises
- Generating conversation questions
- Checking user answers
- Giving feedback
- Explaining user mistakes
- Supporting conversation practice

Database Subsystem

The Database Subsystem will use **PostgreSQL** and **MongoDB**.

PostgreSQL will store structured application data, such as:

- Users
- User profiles
- Learning goals

- Learning plans
- Lessons
- Exercises
- Scores
- Progress records

MongoDB will store flexible AI-related data, such as:

- Conversation history
- AI-generated feedback text
- AI-generated exercise content
- Chat messages

Summary of the System Structure

AI Language Learning System

- Client Subsystem
  - React Frontend
- Backend Subsystem
  - User Service — Spring Boot REST API
  - Learning Service — Spring Boot REST API
  - Progress & Feedback Service — Spring Boot REST API
- GenAI Subsystem
  - GenAI Service — Python + FastAPI + LangChain
- Database Subsystem
  - PostgreSQL Database
  - MongoDB Database

1. Initial System Structure

The system will be divided into four main subsystems:

1. Client Subsystem
2. Backend Subsystem
3. GenAI Subsystem
4. Database Subsystem

Client Subsystem

The Client Subsystem will be implemented using **React**. It provides the user interface for the language learning application.

It allows users to:

- Register and log in
- Set up their language profile
- Choose their target language and learning goal
- View their personalized learning plan
- Solve exercises
- Practice conversations
- View feedback and progress

Backend Subsystem

The Backend Subsystem will be implemented using **Spring Boot REST APIs**. It contains the main application logic and connects the frontend, database, and GenAI service.

For simplicity, the backend will be divided into two Spring Boot microservices:

1. User Service
2. Learning Service

User Service

The User Service manages all user-related functionality.

It is responsible for:

- User registration
- User login
- User profile management
- Saving the target language
- Saving the current language level
- Saving the learning goal
- Saving the deadline or available study time

Learning Service

The Learning Service manages the learning process.

It is responsible for:

- Creating learning plans
- Managing lessons
- Managing exercises
- Storing user answers
- Storing feedback results
- Tracking simple user progress

GenAI Subsystem

The GenAI Subsystem will be implemented as a separate **Python microservice** using **FastAPI and LangChain**.

It is responsible for:

- Generating personalized learning plans
- Generating vocabulary and grammar exercises
- Generating conversation questions
- Checking user answers
- Giving feedback

- Explaining user mistakes

Database Subsystem

The Database Subsystem will use **PostgreSQL** as the main database.

It stores:

- Users
- User profiles
- Learning goals
- Learning plans
- Lessons
- Exercises
- User answers
- Feedback
- Progress records

Summary of the System Structure

```
AI Language Learning System
│
├── Client Subsystem
│   └── React Frontend
│
├── Backend Subsystem
│   ├── User Service        Spring Boot REST API
│   └── Learning Service    Spring Boot REST API
│   └── Progress & Feedback Service    Spring Boot REST API
│
├── GenAI Subsystem
│   └── GenAI Service       Python + FastAPI + LangChain
│
└── Database Subsystem
    └── PostgreSQL
    └── MongoDB Database
```